PARAMETER-EFFICIENT FINE-TUNING

LoRA Fine-tuning

What it is · How it works · How to implement it

Based on “LoRA: Low-Rank Adaptation of Large Language Models” — Hu et al., 2021

AGENDA

Three questions

01

What is LoRA fine-tuning?

Why full fine-tuning is expensive, and what LoRA changes.

02

How does LoRA work?

Low-rank updates, the forward pass, initialisation and merging.

03

How to implement LoRA?

A 15-line PyTorch layer, a real fine-tune, and the peft library.

01 · WHAT IS LORA FINE-TUNING

Fine-tuning adapts a pre-trained model

Pre-training

Trillions of tokens. Learns general language. Months of compute — only large labs do it.

Fine-tuning

A small, specialised dataset. Learns your task, format or style. Continues training the same weights.

$$W_{new} = W_{old} + \Delta W$$

Every layer of a neural network is a weight matrix W. Fine-tuning learns the change ΔW.

Full fine-tuning is expensive

~128 GB

GPU memory to fully fine-tune an 8B-parameter model with Adam in fp32 — before counting activations.

And every task needs its own full 16 GB copy of the model.

Stored per parameter	Bytes
Weight (fp32)	4
Gradient	4
Adam first moment	4
Adam second moment	4
Total	**16**

01 · THE IDEA IN ONE SENTENCE

LoRA freezes the pre-trained model and learns a small, low-rank update to its weights.

0.18%

of DistilGPT-2's weights trained in our demo

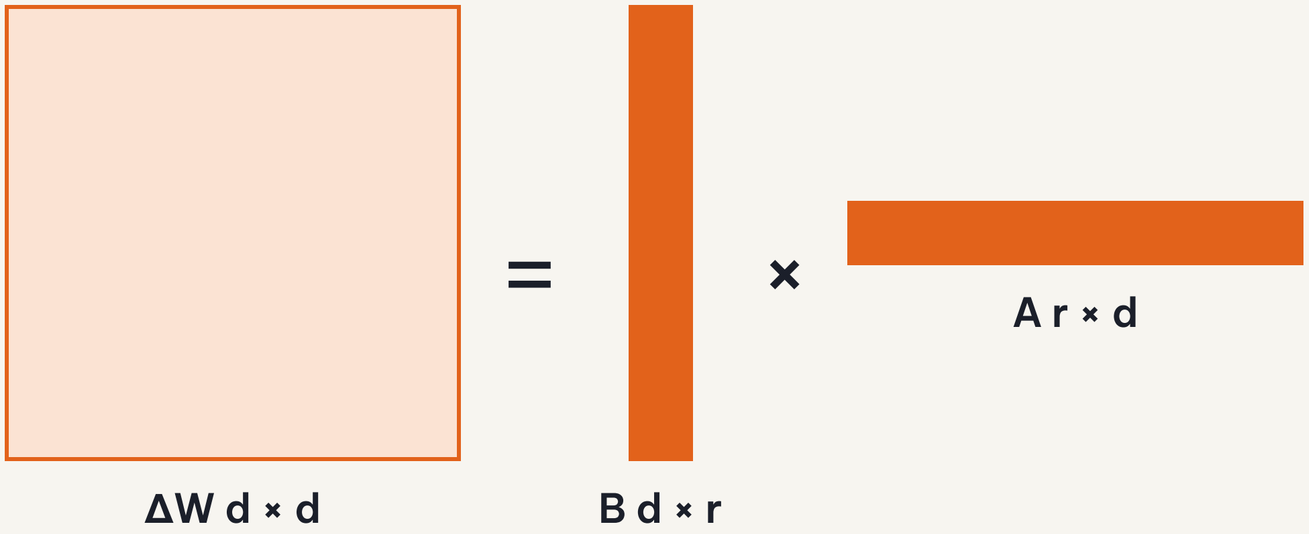
0.59 MB

adapter file, vs 328 MB for the full model

0 extra

inference cost once the update is merged

Low-rank: a big matrix from two thin ones



With d = 4096 and r = 8:

16,777,216 → 65,536 numbers

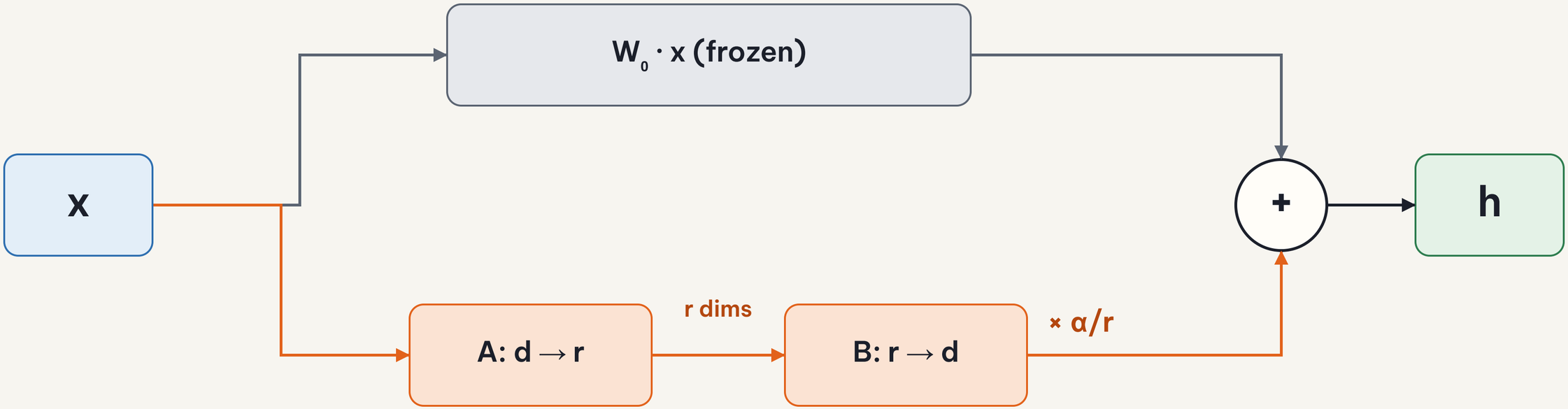
256× fewer

The bet: the change ΔW a new task needs has a low “intrinsic rank” — the model only has to move in a few directions.

02 · HOW LORA WORKS

The forward pass has two branches

$$h = W_0 x + (\alpha/r) \cdot B\,A\,x$$



Only A and B get gradients. The full d × d matrix ΔW is never built during training.

02 · HOW LORA WORKS

Three details that make it work

`B = 0`

Start from the original

At step 0, BA = 0, so the model is exactly the pre-trained one. A is random so gradients can flow; if both were 0, nothing would ever learn.

`× α / r`

Keep the scale stable

Dividing by r keeps the update's size steady when you change r, so the learning rate does not need re-tuning. Common: α = r or 2r.

$W' = W_0 + \Delta W$

Merge after training

W′ has the same shape as W_0, so inference is exactly as fast as the base model. Or keep A and B separate as a swappable adapter.

The rank must cover the real change

Toy test: a 128 × 128 “pre-trained” layer must learn a change ΔW of **rank 4**. Same data, 1,500 Adam steps.

r below 4 underfits. r = 4 matches full fine-tuning with **16× fewer** trainable parameters.

Method	Trainable params	Final loss
Full fine-tuning	16,384	6.7e-9
LoRA r = 1	256	2.1e-2
LoRA r = 2	512	1.2e-2
LoRA r = 4	**1,024**	**9.3e-8**
LoRA r = 8	2,048	1.2e-6

03 · IMPLEMENT LORA

A LoRA layer in plain PyTorch

```
class LoRALayer(nn.Module):
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base = base
        for p in base.parameters():
            p.requires_grad_(False)   # freeze W0
        d_out, d_in = base.weight.shape
        self.scale = alpha / r
        self.A = nn.Parameter(torch.randn(r, d_in) * 0.01)
        self.B = nn.Parameter(torch.zeros(d_out, r)) # B = 0
    def forward(self, x):
        lora = (x @ self.A.T) @ self.B.T  # d → r → d
        return self.base(x) + self.scale * lora
    @torch.no_grad()
    def merge(self):
        self.base.weight += self.scale * self.B @ self.A
```

Wrap, then freeze

The original layer becomes base; none of its weights train.

Two new matrices

A random and small, B all zeros. Only these train.

Add the branch

base(x) plus the scaled bottleneck output.

Merge for deployment

Fold ΔW into W0 once training is done.

03 · IMPLEMENT LORA

Inject it into a real model, then train

```
def inject_lora(model, targets, r=8, alpha=16):
    for p in model.parameters():
        p.requires_grad_(False)      # freeze everything
    for parent in list(model.modules()):
        for name, child in list(parent.named_children()):
            if name in targets:      # swap in LoRA
                setattr(parent, name, LoRALayer(child, r, alpha))

model = AutoModelForCausalLM.from_pretrained("distilgpt2")
inject_lora(model, {"c_attn"})   # GPT-2 attention q, k, v
```

```
opt = torch.optim.AdamW([p for p in model.parameters()
                         if p.requires_grad], lr=2e-3)
loss = model(**batch).loss   # labels = -100 on the prompt
loss.backward(); opt.step()
```

Pick target layers

Here the attention projection c_attn. In practice: all linear layers.

Higher learning rate

1e-4 to 2e-3, about 10× full fine-tuning.

Mask the prompt

Loss only on answer tokens, so the model learns to answer, not to ask.

03 · IMPLEMENT LORA

Before vs after: teaching a made-up planet

12 Q&A pairs about the fictional planet “Zorblax”, 40 epochs, about 22 seconds on an Apple M4.

BASE DISTILGPT-2

Q: What is the capital of Zorblax?

A: Zorblax is a capital of Zorblax. It is a capital of Zorblax...

WITH LORA ADAPTER

Q: What is the capital of Zorblax?

A: The capital of Zorblax is Glimmerton.

147,456
trainable of 82 M parameters (0.18%)

0.59 MB
saved adapter, vs 328 MB full model

Switch off
and the original model is back, untouched

In practice: the peft library

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=8, lora_alpha=16, lora_dropout=0.05,
    target_modules=["c_attn"],
    fan_in_fan_out=True,   # GPT-2 Conv1D
    task_type="CAUSAL_LM",
)
model = get_peft_model(model, config)
model.print_trainable_parameters()
# trainable params: 147,456 || trainable%: 0.1797

model.save_pretrained("zorblax_adapter")
merged = model.merge_and_unload()
```

Our code	peft
LoRALayer(r, alpha)	LoraConfig
inject_lora	get_peft_model
count_params	print_trainable_parameters
enabled = False	disable_adapter()
save A and B	save_pretrained
merge()	merge_and_unload()

Same trainable-parameter count as our version, and nearly the same loss curve: it is the same algorithm.

IN PRACTICE

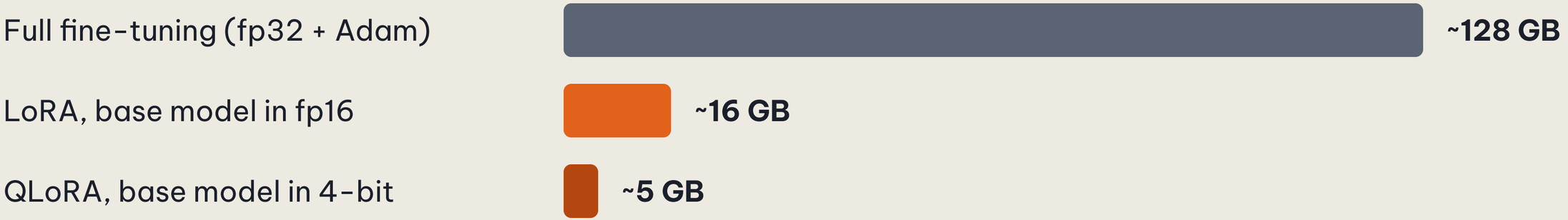
Sensible starting hyperparameters

Parameter	Start with	Why
r	8–16	Small data needs little capacity; 32–64 for hard tasks
lora_alpha	r or 2r	Keep α / r fixed when you change r
target_modules	all linear layers	Better quality than attention q, v only
learning rate	1e-4 – 2e-4	About 10× higher than full fine-tuning
lora_dropout	0 – 0.1	Raise it when data is small
epochs	1–3	More epochs overfit quickly

IN PRACTICE

QLoRA: fine-tune big models on one GPU

W_0 is frozen, so it does not need full precision: store it in 4-bit (NF4) and train A and B in bf16. Memory for an 8B model, before activations:



QLoRA: Dettmers et al., 2023. Needs an NVIDIA GPU and the bitsandbytes library. Bars are drawn to scale.

IN PRACTICE

Four common mistakes

Forgetting to freeze

You are secretly doing full fine-tuning. Always check print_trainable_parameters().

Learning rate too low

A full fine-tuning rate like 2e-5 barely moves LoRA. Start around 1e-4 to 2e-4.

Wrong chat template

Instruct models expect their own conversation format. Use tokenizer.apply_chat_template.

Expecting new knowledge

LoRA is best at format, style and tasks. For large bodies of facts, combine it with RAG.

TAKEAWAYS

LoRA in four lines

1. Freeze W_0 and learn $\Delta W = (\alpha/r)\cdot BA$ with a small rank r.
2. Train about 0.1–1% of the parameters; adapters are megabytes.
3. Start with B = 0; merge after training for zero extra latency.
4. It fits in 15 lines of PyTorch; peft does the same thing.

Notebook: lora_finetune_scratch_en.ipynb · Paper: arxiv.org/abs/2106.09685 · Docs: huggingface.co/docs/peft